

database\compat.py

```
from database.db_manager import DatabaseManager
from datetime import datetime
import json

# Add missing methods to DatabaseManager
def get_cache_metadata(self, key):
    """Get cache metadata with TTL check"""
    if not self.use_database:
        # Return default metadata in memory mode
        return {
            'last_updated': datetime.now().isoformat(),
            'total_records': 0,
            'metadata': {},
            'is_cache': False
        }

    conn = self.get_connection()
    if not conn:
        print(f"[Database] Failed to get connection for get_cache_metadata: {key}")
        return None

    try:
        cursor = conn.cursor()

        # Check if cache_metadata table exists
        if 'cache_metadata' not in self.cached_database_tables:
            # Create the table if it doesn't exist
            cursor.execute("""
            CREATE TABLE IF NOT EXISTS cache_metadata (
            key VARCHAR(100) PRIMARY KEY,
            last_updated TIMESTAMP,
            total_records INTEGER,
            metadata JSONB
            )
            """)
            conn.commit()
            self.cached_database_tables['cache_metadata'] = True

        cursor.execute("""
        SELECT last_updated, total_records, metadata
        FROM cache_metadata
        WHERE key = %s
        """, (key,))

        row = cursor.fetchone()
        if row:
            # Check TTL (24 hours to reduce database load)
            last_updated = row[0]
            total_records = row[1]
            metadata = row[2]

            return {
                'last_updated': last_updated,
                'total_records': total_records,
                'metadata': metadata,
                'is_cache': True
            }

        # If no data found or cache expired, return default structure
        return {
            'last_updated': datetime.now().isoformat(),
            'total_records': 0,
            'metadata': {},
            'is_cache': False
        }

    except Exception as e:
        print(f"[Database] Error getting cache metadata: {str(e)}")
        return {
            'last_updated': datetime.now().isoformat(),
            'total_records': 0,
            'metadata': {},
        }
```

```

'is_cache': False
}

finally:
if 'cursor' in locals():
    cursor.close()
self.return_connection(conn)

def save_cache_metadata(self, key, total_records, metadata):
    """Save cache metadata with TTL"""
    if not self.use_database:
        return False

    conn = self.get_connection()
    if not conn:
        print(f"[Database] Failed to get connection for save_cache_metadata: {key}")
        return False

    try:
        cursor = conn.cursor()

        # Check if cache_metadata table exists
        if 'cache_metadata' not in self.cached_database_tables:
            # Create the table if it doesn't exist
            cursor.execute("""
            CREATE TABLE IF NOT EXISTS cache_metadata (
            key VARCHAR(100) PRIMARY KEY,
            last_updated TIMESTAMP,
            total_records INTEGER,
            metadata JSONB
            )
            """)
            conn.commit()
            self.cached_database_tables['cache_metadata'] = True

        cursor.execute("""
        INSERT INTO cache_metadata (key, last_updated, total_records, metadata)
        VALUES (%s, CURRENT_TIMESTAMP, %s, %s)
        ON CONFLICT (key) DO UPDATE SET
        last_updated = CURRENT_TIMESTAMP,
        total_records = EXCLUDED.total_records,
        metadata = EXCLUDED.metadata
        """, (key, total_records, json.dumps(metadata)))

        conn.commit()
        return True

    except Exception as e:
        print(f"[Database] Error saving cache metadata: {str(e)}")
        conn.rollback()
        return False

    finally:
        if 'cursor' in locals():
            cursor.close()
        self.return_connection(conn)

# Patch the DatabaseManager class with the missing methods
DatabaseManager.get_cache_metadata = get_cache_metadata
DatabaseManager.save_cache_metadata = save_cache_metadata

# Print confirmation
print("[Database] Added compatibility methods to DatabaseManager")

```